

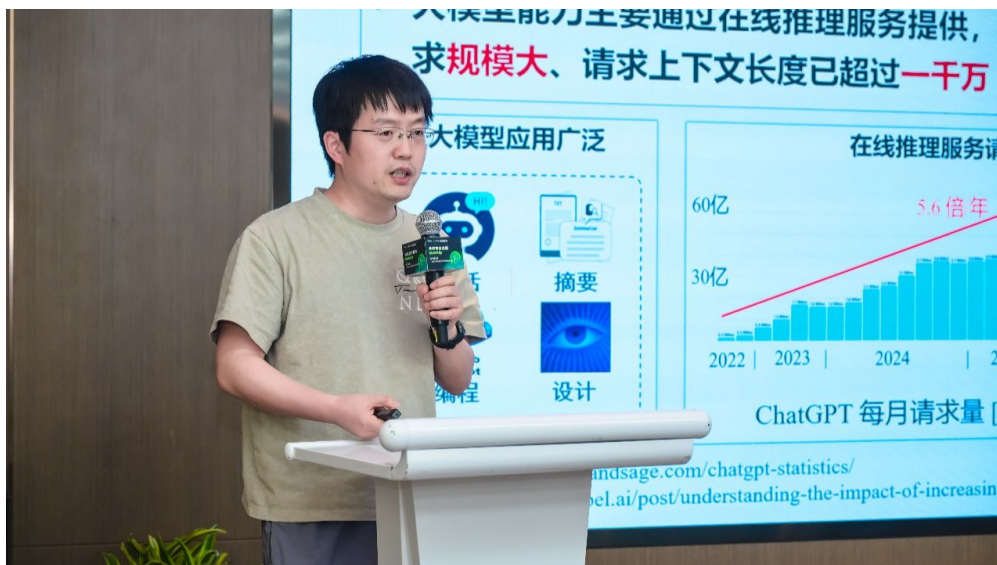
课程笔记

面向 multi-agent 场景的长上下文管理方法

白有辉 (USTC-MLSys)

SSS & Codex 整理

2026 年 6 月 18 日



视频作者/频道: 龙蜥社区 _ 小龙

发布日期: Bilibili 发布: 2026 年 6 月 12 日; 原汇报: 2026 年 1 月 28 日

视频时长: 19 分 22 秒

视频链接: <https://www.bilibili.com/video/BV1nHEy6rEEP/>

目录

阅读导引	2
1 multi-agent 场景中的长上下文压力	2
1.1 多智能体交互如何制造更长的上下文	2
1.2 前端上下文压缩与后端推理管理的分工	3
1.3 为什么这件事进入系统层	4
1.4 本章小结	5
2 KVCache: 长上下文推理的资源账本	5
2.1 prefill 与 decoding 的分工	5
2.2 KVCache 为什么随序列长度线性增长	5
2.3 HBM 容量与带宽如何形成 memory wall	7
2.4 CPU offload 的收益和 PCIe 成本	7
2.5 本章小结	8
3 稀疏注意力: 只读真正相关的历史	9
3.1 从红楼梦类比理解 token 选择	9
3.2 QK score 与 top-k KV 选择	9
3.3 naive top-k 为什么仍然昂贵	10
3.4 训练无关 KV 检索系统与训练式稀疏注意力	11
3.5 本章小结	11
4 层次化 KVCache: 把容量放在 CPU, 把热点留在 GPU	11
4.1 CPU 全量存储与 GPU 热点缓存	11
4.2 一次 decoding step 的四步路径	11
4.3 metadata 为什么成为隐藏成本	12
4.4 本章小结	13
5 现有系统慢在哪里: metadata 与调度	13
5.1 问题一: 动态 KVCache 管理开销过高	14
5.2 LRU/LFU 与 GPU 并发执行的结构冲突	15
5.3 缓存粒度: 粗粒度牺牲选择质量, 细粒度膨胀 metadata	15
5.4 问题二: CPU-GPU 交互破坏 CUDA Graph 捕获	16
5.5 本章小结	17
6 轻量缓存算法: 用 query 相似性替代链表 metadata	17
6.1 观察: 相邻 decoding step 的 query 高度相似	17
6.2 从 query 相似到 top-k KV 重合	18
6.3 核心算法: 阈值命中与上一轮 top-k 复用	19
6.4 自适应阈值、resident head 与 GQA 聚合	20
6.5 本章小结	21

7	协同调度：让数据搬运尽量隐藏在计算之后	21
7.1	miss 数据仍需搬运：算法无法消灭物理带宽	22
7.2	与 InfiniGen 类 overlap 思想的结合	22
7.3	GPU 触发 CPU：恢复 CUDA Graph 友好执行	23
7.4	GDRCopy 与额外拷贝消除	23
7.5	本章小结	24
8	实验评估：性能、容量、精度和显存	24
8.1	实验平台与基线系统	24
8.2	decoding throughput 与可支持上下文长度	25
8.3	metadata 开销拆解	26
8.4	LongBench/RULER 精度与 FullAttn 可比性	27
8.5	GPU 显存占用与 Prefill 影响	27
8.6	本章小结	27
9	总结与延伸	28
9.1	一句话压缩	28
9.2	三层问题：容量、检索、调度	28
9.3	方法适用边界	29
9.4	对 multi-agent 推理系统的启发	29
9.5	开放问题	29
9.6	复盘检查表	30

阅读导引

阅读导引

这份笔记只处理视频中最核心的系统问题：multi-agent 长上下文推理如何管理 KVCache。全篇主线是：需求增长先制造容量压力，容量压力引出 CPU offload，offload 又带来 KV 检索、metadata 管理和 CPU-GPU 协同问题；讲者提出的方案则用 query similarity 降低缓存管理成本，并通过 GPU-centric 协同和 GDRCopy 降低数据搬运的可见开销。

1 multi-agent 场景中的长上下文压力

1.1 多智能体交互如何制造更长的上下文

讲者进入主题时先把问题放在一个现实背景里：大模型正在被放进更复杂的任务系统，输入序列长度随之持续变长。multi-agent（多智能体）场景会把这个趋势进一步放大，因为一次任务很少只由一个模型调用完成。常见流程会包含规划 agent、执行 agent、代码或检索工具、审查 agent、修复 agent，以及下一轮反思。每一轮都会把任务目标、已做决策、工具输出、失败原因和新约束追加进上下文。

¹视频画面时间区间：00:00:10–00:00:18。若为裁剪图，时间区间仍对应原视频画面。



图 1: 标题页确认了课程题目、汇报人白有辉、课题组 USTC-MLSys、团队负责人李诚特任教授和汇报时间。¹

一个直观类比是多人接力写实验记录。第一个人记录实验目的，第二个人补上测量数据，第三个人指出异常，第四个人写修正方案。为了让后来者不重复踩坑，记录本会越写越厚。multi-agent 系统中的上下文也类似：它保存的已经超出原始用户请求，还包含系统运行过程中的状态轨迹。

从“长提示词”到“长期任务状态”

在 multi-agent 场景里，长上下文更接近一个不断增长的任务状态表：包含原始需求、角色分工、工具结果、错误日志、阶段性结论和下一步约束。若这些状态丢失，后续 agent 会重新探索；若全部保留，推理系统要承受更高的 token 和 KVCache 压力。

讲者提到 1 million token 量级的上下文能力，目的在于说明行业正在把长上下文从“少数模型特性”推向“系统默认假设”。这个趋势对 multi-agent 尤其关键：单个 agent 对话也许可以通过手工删减历史继续运行，多 agent 接力一旦频繁丢失状态，就会出现重复调用工具、重复生成方案、重复犯错等问题。

1.2 前端上下文压缩与后端推理管理的分工

讲者随后把“长上下文管理”拆成两层。第一层发生在推理请求形成之前，可以称为前端上下文压缩或 context compaction / summary。它回答的问题是：系统应该把哪些历史内容压缩成当前请求的一部分？比如在 coding agent 或 Web coding 环境里，一个对话框到达上下文限制后，系统会把已有对话、工具结果和任务状态做一次摘要，再带着这个摘要继续。

第二层发生在大模型推理框架内部。前端已经决定了这次请求要喂给模型多少 token；后端需要处理这些 token 在 Transformer 推理中的存储、读取、搬运和计算。讲者明确说明本次分享主要关注后者，也就是推理框架如何管理长序列请求产生的 KVCache。

²视频画面时间区间：00:00:20–00:01:50。若为裁剪图，时间区间仍对应原视频画面。

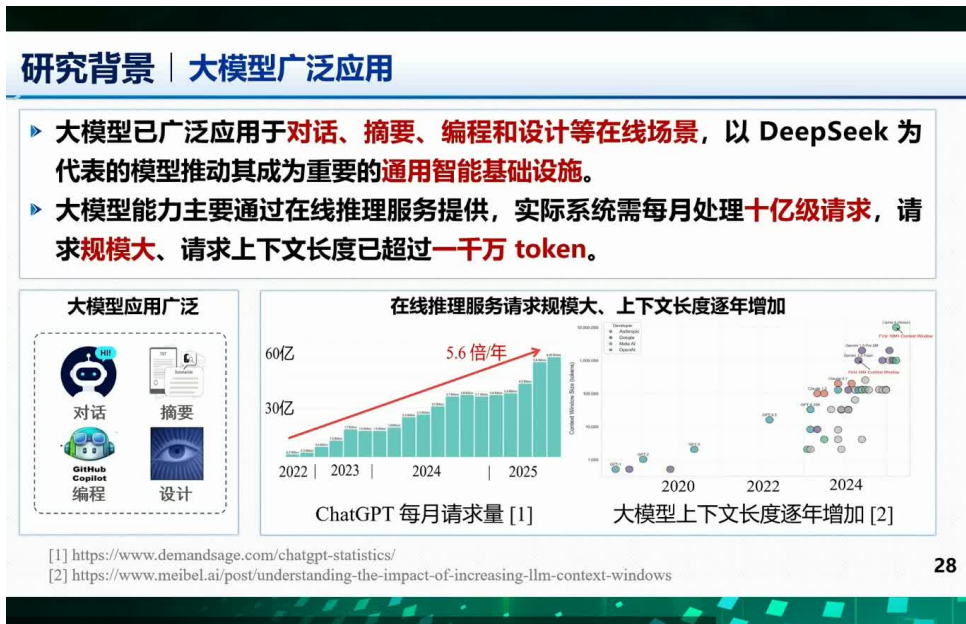


图 2: multi-agent 与在线推理场景推动上下文长度进入百万 token 量级，系统层管理由此成为关键问题。²

本章的边界

前端上下文压缩解决“哪些历史信息应该进入下一次请求”；后端 KVCache 管理解决“进入请求后的长序列如何在 GPU、CPU 和缓存结构中被高效存取”。两者共同影响 multi-agent 系统的可用性，但技术对象完全不同。

这个分工很重要。若把两层混在一起，容易误以为只要摘要做得好，推理系统就没有长上下文压力。实际情况更硬：即使前端已经做过摘要，后端单次请求仍可能达到 200K token 甚至更长。模型每处理一次这样的请求，都要在每层、每个相关 head 中生成并维护对应的 key/value 状态。

1.3 为什么这件事进入系统层

multi-agent 系统对长上下文的需求来自任务语义，系统层压力来自硬件账本。语义上，任务状态需要延续；硬件上，长序列会变成 GPU 显存中的 KVCache、HBM 带宽读取、CPU-GPU 数据搬移以及缓存元数据维护。前者像项目管理，后者像仓储和物流。一个项目记录得再清楚，仓库容量、货架索引和搬运通道仍然会限制吞吐。

从系统设计角度看，长上下文管理至少牵涉三个层面：

- **容量层**：请求长度 L 增长时，KVCache 近似线性增长，GPU HBM 很快成为稀缺资源。
- **访问层**：decoding 阶段每生成一个 token 都要访问历史 KV；长上下文会把读取历史的成本放大。
- **调度层**：当 KVCache 被卸载到 CPU，GPU 与 CPU 之间的查表、加载、更新和同步会进入关键路径。

容易漏掉的系统事实

上下文窗口扩展并不等价于推理成本自动可承受。窗口像图书馆允许借阅的最大书架长度，KVCache 管理像每次回答问题时实际取书、查目录、搬书和归位的过程。窗口越大，后端系统越需要精细地控制哪些历史真的被读取、哪些数据留在 GPU、哪些数据卸载到 CPU。

1.4 本章小结

- multi-agent 会把任务目标、工具结果、反思和约束不断追加进上下文，长上下文压力由此被放大。
- 上下文管理分成前端 summary / compaction 和后端推理框架内部管理两层，本视频主要讨论后端。
- 即使前端压缩已经发生，单次推理请求仍可能达到 200K token 以上，后端依然要面对 KVCache 的容量和访问成本。
- 从下一章开始，问题会落到更具体的资源账本：prefill 如何生成 KVCache，decoding 如何追加 KV，以及 HBM 与 PCIe 如何形成瓶颈。

2 KVCache：长上下文推理的资源账本

2.1 prefill 与 decoding 的分工

进入推理框架后，讲者把大模型推理拆成两个阶段：prefill 和 decoding。prefill 阶段一次性处理输入 prompt，把输入序列中每个 token 在每一层产生的 key 和 value 保存下来；decoding 阶段进入自回归生成，每一步生成一个新 token，并把这个新 token 对应的 K, V 继续追加到历史 KVCache 中。

可以把 prefill 看成把一本参考书先做成索引，把 decoding 看成每写一个新字都要回头查这本索引。prompt 越长，索引越大；生成越久，索引继续增长。Transformer 的注意力机制需要当前 query 去访问历史 key/value，因此 KVCache 是加速 decoding 的必要账本。

KVCache 的角色

KVCache 保存历史 token 在各层中的 key/value 表示。它用显存换计算：decoding 时无需反复重算历史 token 的 key/value，但必须持续保存、读取并追加这些状态。

2.2 KVCache 为什么随序列长度线性增长

KVCache 的容量可以用一个近似公式理解。它表达的是：每个 token、每一层、每个 KV head 都要保存一份 key 和一份 value。

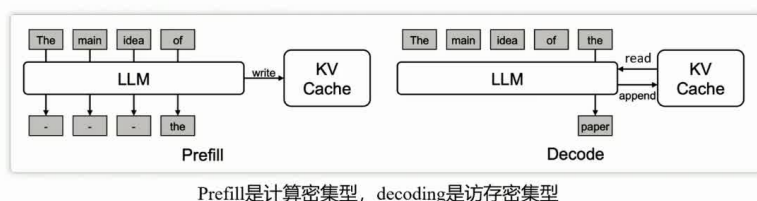
$$\text{KVBytes} \approx 2 \cdot B \cdot L \cdot N_{\text{layer}} \cdot H_{\text{kv}} \cdot d_h \cdot \text{bytes_per_elem}$$

其中：

³视频画面时间区间：00:02:00–00:02:20。若为裁剪图，时间区间仍对应原视频画面。

研究背景 | 大模型推理流程

- ▶ **大模型推理**：模型从输入文本生成目标输出的过程，是对话交互、内容生成、逻辑推演等下游应用任务的核心环节。
- ▶ **推理过程的两阶段**：Prefill阶段并行处理输入序列，构建KVCache，产生首token；decode阶段重复多轮迭代逐步生成新文本，每次迭代追加最新token的KV，并读取全量历史KVCache。



29

图 3: Prefill 构建初始 KVCache，Decoding 自回归生成 token 并追加新的 KV。³

- 2: 同一个 token 需要保存 key 和 value 两类向量。
- B : batch size, 同时参与推理的请求或样本数量。
- L : 当前上下文 token 长度, 包含输入 prompt 与已经生成的 token。
- N_{layer} : Transformer 层数。
- H_{kv} : KV head 数; 在 GQA 模型中它可能小于 query head 数。
- d_h : 单个 head 的向量维度。
- bytes_per_elem: 单个数值元素占用的字节数, 例如 FP16 通常为 2。

这个公式的工程含义很直接：当 L 翻倍，KVCache 近似翻倍；当 B 翻倍，KVCache 也近似翻倍。模型层数、head 数和精度格式固定后，长上下文最先把压力投到 L 和 B 上。

为什么公式里是 H_{kv}

很多现代模型采用 GQA (Grouped-Query Attention)，多个 query head 共享一组 KV head。这样可以降低 KVCache 容量，因为公式中的 H_{kv} 小于 query head 数 H_q 。这只能降低常数项，线性增长的主趋势仍由 B 和 L 决定。

GPU HBM 可用容量还要同时容纳模型权重、激活临时空间、运行时 workspace 和其他缓存。一个更贴近部署的约束可以写成：

$$\text{KVBytes} \leq M_{\text{HBM}} - M_{\text{weights}} - M_{\text{workspace}} - M_{\text{runtime}}$$

其中 M_{HBM} 是可用 GPU 显存， M_{weights} 是模型权重占用， $M_{\text{workspace}}$ 是算子临时空间， M_{runtime} 是运行时和碎片化带来的额外占用。这个不等式说明，长上下文成本同时取决于模型标称窗口和实际部署时剩余显存。

⁴ 视频画面时间区间：00:02:20–00:03:20。若为裁剪图，时间区间仍对应原视频画面。

研究背景 | 大模型推理面临的内存墙问题

- ▶ **KVCache内存体积随序列长度、批次请求数量而线性增长**，长文本应用的出现与流行加剧了该问题。
- ▶ 然而，GPU等AI计算卡内存（HBM）昂贵，容量增长速度远远落后于模型规模、序列长度所需的内存增长速度。

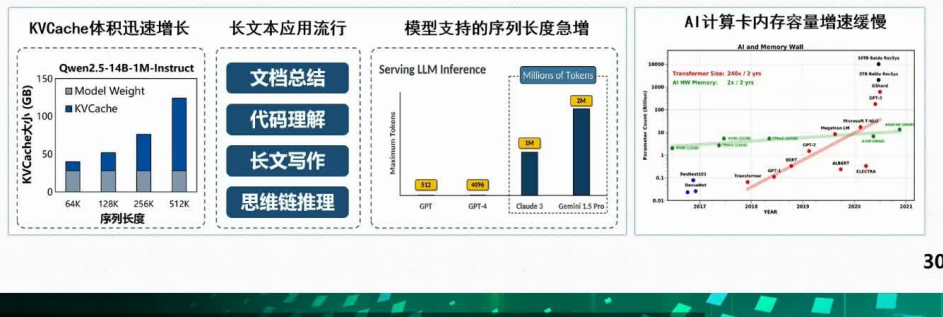


图 4: KVCache 容量随序列长度和 batch size 增长，HBM 容量和带宽共同形成 memory wall。⁴

2.3 HBM 容量与带宽如何形成 memory wall

容量只是第一层问题。讲者接着强调 decoding 还是 memory-bound / bandwidth-bound 的计算：每生成一个 token，注意力需要从 HBM 中读取大量历史 KV。若历史长度很长，GPU 的计算单元可能并没有被算术操作塞满，瓶颈却出现在“把 KV 从显存读出来”这一步。

一个粗略的读取时间模型是：

$$T_{\text{read}} \approx \frac{\text{BytesRead}}{BW_{\text{HBM}}}$$

其中 BytesRead 是这一步需要读取的 KV 字节数， BW_{HBM} 是 HBM 的有效带宽。长上下文使 BytesRead 增大，memory wall 的本质就是带宽增长追不上 KVCache 访问需求增长。

容量墙与带宽墙是两件事

HBM 容量不足时，系统放不下完整 KVCache；HBM 带宽不足时，系统放得下也读得慢。长上下文推理经常同时遇到这两类瓶颈，因此只扩显存容量无法自动解决 decoding latency。

讲者还指出一个更长期的趋势：硬件内存容量的增长较慢，而模型参数量和 KVCache 需求会随模型规模、上下文长度和 batch size 快速增长。于是 KVCache 成为长上下文推理中最突出的 memory wall。

2.4 CPU offload 的收益和 PCIe 成本

第一类自然方案是 CPU offload：把全量或大部分 KVCache 放到 CPU memory 中，GPU HBM 只保留当前计算需要的数据或热点数据。它利用了单机服务器中 CPU 内存容量远大于 GPU HBM 的事实，可以缓解“放不下”的问题。

代价出现在数据通道上。CPU 与 GPU 通常通过 PCIe 互联，PCIe 有效带宽远低于 HBM。若某一步 decoding 需要从 CPU 搬回大量 KV，则可见搬运时间近似为：

$$T_{\text{copy}} \approx \frac{\text{BytesMiss}}{BW_{\text{PCIe}}}$$

其中：

- BytesMiss：本步在 GPU 上未命中、必须从 CPU memory 搬回的 KV 字节数。
- BW_{PCIe} ：CPU-GPU 之间 PCIe 链路的有效带宽。
- T_{copy} ：这部分搬移对 decoding step 造成的时间成本。

讲者给出的理论测算指出，随着序列长度上升，GPU 上的计算时间与 CPU 侧通过 PCIe 搬运 KV 的时间相比会变得很小，极端情况下 90% 以上时间可能花在搬 KV 上。这里应重视的是量级含义：CPU offload 把容量问题转成了跨设备搬运问题。

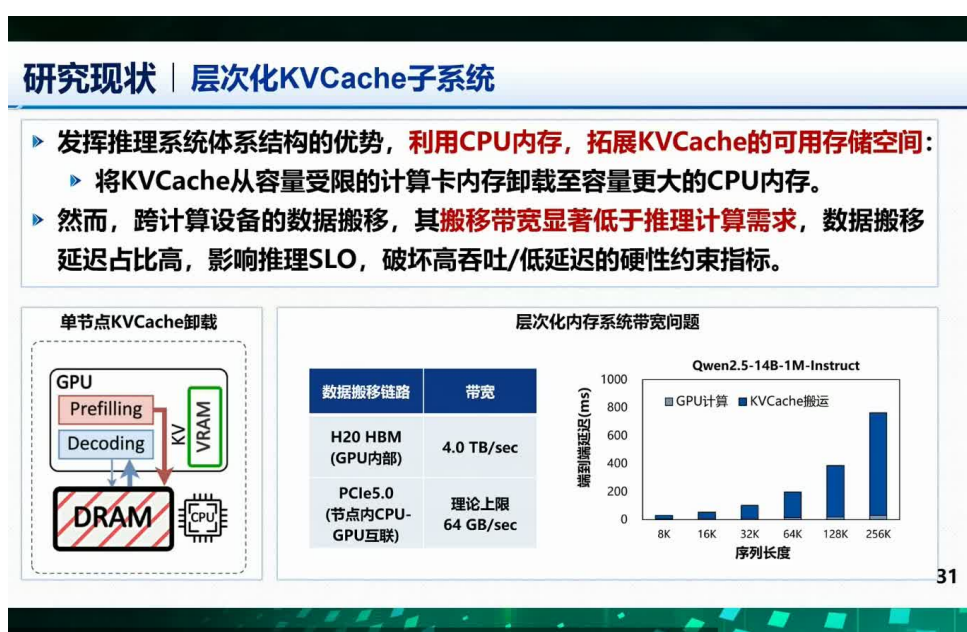


图 5: CPU memory 提供容量扩展，PCIe 搬运成为长上下文 Decoding 的主要延迟来源之一。⁵

2.5 本章小结

- prefill 生成初始 KVCache，decoding 每步自回归生成 token 并追加新的 KV。
- KVCache 容量近似为 $2BLN_{\text{layer}}H_{\text{kv}}d_h\text{bytes_per_elem}$ ，随 B 和 L 线性增长。
- HBM 的问题包含容量和带宽两层：放不下是容量墙，读得慢是 bandwidth-bound 的 memory wall。
- CPU offload 能缓解容量压力，但 PCIe 搬运会进入 decoding 的关键路径。
- 下一章的稀疏注意力要解决的核心是减少每步真正需要读取和搬运的历史 KV。

⁵视频画面时间区间：00:03:30-00:04:20。若为裁剪图，时间区间仍对应原视频画面。

3 稀疏注意力：只读真正相关的历史

3.1 从红楼梦类比理解 token 选择

在 CPU offload 暴露 PCIe 成本之后，讲者引出第二条路线：sparse attention（稀疏注意力）。它的核心观察是，长上下文推理中的一次生成通常只依赖历史 KV 中的一部分 token。当前问题像一个检索意图，历史上下文像一本很厚的书；合理做法是找到相关章节，再基于这些章节回答。

讲者用《红楼梦》的例子说明这个直觉：若问题是“贾宝玉和林黛玉什么时候相遇”，读者会带着这个问题定位相关章节，再整理答案。逐字重读全书会浪费大量时间。映射到注意力机制里，当前 query 也在“询问”历史 token，系统希望挑出贡献最大的历史 KV，避免每一步都读取完整历史。

稀疏注意力的直觉

稀疏注意力让每一步生成把访问预算集中到最相关的历史 token 上。它像带索引的查书：全书仍在，当前问题只翻最可能有答案的页。

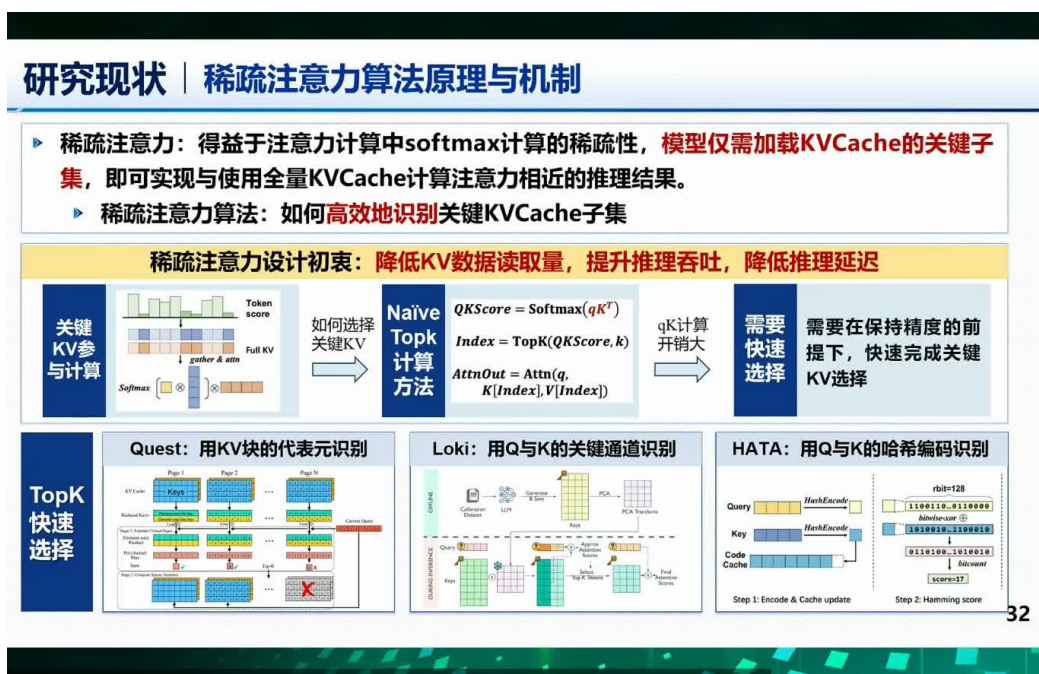


图 6: 稀疏注意力尝试识别关键 KV 子集，Quest、Loki、HATA 等方法围绕更快的 top-k 选择展开。

6

3.2 QK score 与 top-k KV 选择

标准 attention 会先计算 query 和各个历史 key 的相关性分数。对第 t 个 decoding step、第 h 个 head，可以写成：

$$s_{t,i}^{(h)} = \frac{q_t^{(h)} \cdot k_i^{(h)}}{\sqrt{d_h}}$$

其中：

⁶ 视频画面时间区间：00:04:30–00:06:20。若为裁剪图，时间区间仍对应原视频画面。

- $q_t^{(h)}$: 当前 step t 、head h 的 query 向量。
- $k_i^{(h)}$: 历史位置 i 、head h 的 key 向量。
- d_h : 单个 head 的向量维度。
- $s_{t,i}^{(h)}$: 当前 query 对历史位置 i 的 attention score。
- $\sqrt{d_h}$: 缩放因子, 用于稳定点积分数的数值范围。

朴素 top-k 稀疏化会选择分数最高的 K_{top} 个历史位置:

$$S_t^{(h)} = \text{TopK}_{i \leq L} \left(s_{t,i}^{(h)}, K_{\text{top}} \right)$$

这里 $S_t^{(h)}$ 是本 step、本 head 选中的 top-k KV index 集合, K_{top} 是需要保留的历史 token 数。后续 attention 只读取这些位置上的 value, 并对它们做加权求和:

$$\text{AttnOut}_t^{(h)} \approx \sum_{i \in S_t^{(h)}} \alpha_{t,i}^{(h)} v_i^{(h)}$$

其中 $v_i^{(h)}$ 是历史位置 i 的 value, $\alpha_{t,i}^{(h)}$ 是在选中集合内归一化后的注意力权重。这个近似的收益来自两处: value 读取量减少, 后续 attention 计算也减少。

top-k KV 的系统意义

top-k KV index 是算法和系统之间的接口。算法负责判断当前 step 最需要哪些历史位置; 系统负责把这些位置的 KV 从 GPU cache 或 CPU memory 中取到计算路径上。

3.3 naive top-k 为什么仍然昂贵

朴素 top-k 的问题藏在“找到 top-k”这一步。为了知道哪些 key 分数最高, 系统可能仍要把所有历史 key 扫描一遍, 计算所有 $q_t \cdot k_i$ 。这会保留一项随 L 增长的 key 扫描成本。

减少 value 读取不等于消除 top-k 检索成本

稀疏 attention 可以减少 value 的加载量和后续 attention 计算量; 寻找 top-k key 本身仍可能需要遍历完整历史 key。若 top-k 检索阶段要扫描全部 K , 检索成本会成为新的瓶颈。系统优化必须同时回答两个问题: 读哪些 value, 以及怎样低成本找到这些 value 对应的 key index。

这个警告是理解后续系统工作的关键。若系统只把 attention 的 value 读取变稀疏, 却没有降低 key 检索成本, 长上下文下的主要压力会从 value 侧转移到 key 扫描和 top-k 排序侧。换句话说, 稀疏注意力需要一个高效 KV 检索系统配合, 单靠“少读 value”这一条收益不足以支撑系统级加速。

可以用一个粗略表达式拆开成本:

$$T_{\text{sparse}} \approx T_{\text{find_topk}} + T_{\text{read_V(topk)}} + T_{\text{attn(topk)}}$$

其中 $T_{\text{find_topk}}$ 是寻找 top-k key 的时间, $T_{\text{read_V(topk)}}$ 是读取选中 value 的时间, $T_{\text{attn(topk)}}$ 是稀疏 attention 计算时间。若 $T_{\text{find_topk}}$ 随完整历史长度 L 线性增长, 系统仍会在长上下文下变慢。

3.4 训练无关 KV 检索系统与训练式稀疏注意力

针对 naive top-k 的全 key 扫描问题，业界提出了一批训练无关的 KV 检索方法。高清帧中可以清楚看到 Quest、Loki、HATA 等路线，它们的共同目标是更快地选择出 top-k KV index，让系统在不重新训练模型的情况下减少检索成本。讲者口头还提到其他相关工作，正文不展开未在画面中清楚呈现的缩写，避免把 ASR 误识别写成确定结论。

另一类路线进入训练结构本身。讲者提到 DeepSeek 相关的稀疏 attention 设计，其要点是把选择式注意力纳入模型训练或结构设计中，使模型在训练阶段就适应稀疏访问模式。字幕里的缩写不稳定，因此这里保留机制层面的解释：训练式路线把“如何选择重要 KV”内化到模型结构或训练过程，训练无关路线则试图在已有模型外部设计更快的检索器。

3.5 本章小结

- 稀疏注意力的出发点是当前 query 通常只需要历史 KV 中少量重要 token。
- QK score 给出 query 与历史 key 的相关性，top-k 选择把相关性最高的位置交给后续 attention。
- 朴素 top-k 可能仍要扫描全部历史 key，因此减少 value 读取无法自动消除检索开销。
- Quest、Loki、HATA 等训练无关方法尝试降低 top-k index 选择成本；DeepSeek 相关路线则把稀疏访问纳入训练结构。
- 下一章会把 CPU offload 和 sparse attention 合在一起，进入层次化 KVCache 系统的执行路径。

4 层次化 KVCache：把容量放在 CPU，把热点留在 GPU

4.1 CPU 全量存储与 GPU 热点缓存

讲者在这一段把前两章的两条路线合并起来：CPU offload 解决容量，sparse attention 减少每一步需要访问的 KV 数量。组合后的系统通常采用层次化 KVCache：全量 KVCache 存在 CPU memory 中，GPU HBM 只保留一部分热点 KV 或近期高价值 KV。

这种结构像把完整档案放在库房，把常用资料放在办公桌。CPU memory 是大库房，容量更大；GPU HBM 是办公桌，离计算最近。系统希望每次 decoding 都尽量在办公桌上拿到需要的材料，只有未命中时再去库房搬。

层次化 KVCache 的基本结构

CPU memory 保存全量 KVCache，on-GPU cache 保存热点 KV。sparse attention 或 KV 检索算法先给出当前 step 的 top-k KV index，系统再判断这些 index 是否已经在 GPU cache 中，未命中的部分通过 CPU-GPU 通道异步加载。

4.2 一次 decoding step 的四步路径

讲者把现有系统抽象成四个稳定步骤。这个抽象很重要，因为后续章节关于 metadata overhead 和 CPU-GPU coordination 的问题，都来自这条路径。

⁷ 视频画面时间区间：00:06:30-00:07:50。若为裁剪图，时间区间仍对应原视频画面。

研究现状 | 稀疏注意力机制驱动的推理系统

- ▶ 稀疏注意力可与层次化KVCache系统结合，降低从CPU内存的数据读取量，**驱动形成高吞吐/低延迟的推理系统**
- ▶ KVCache划分成block进行管理，全量KVCache卸载到CPU内存之后GPU内存出现一定富裕，可用于**缓存热点**topk KV数据，传统方法使用操作系统中常用的LRU/LFU等缓存算法

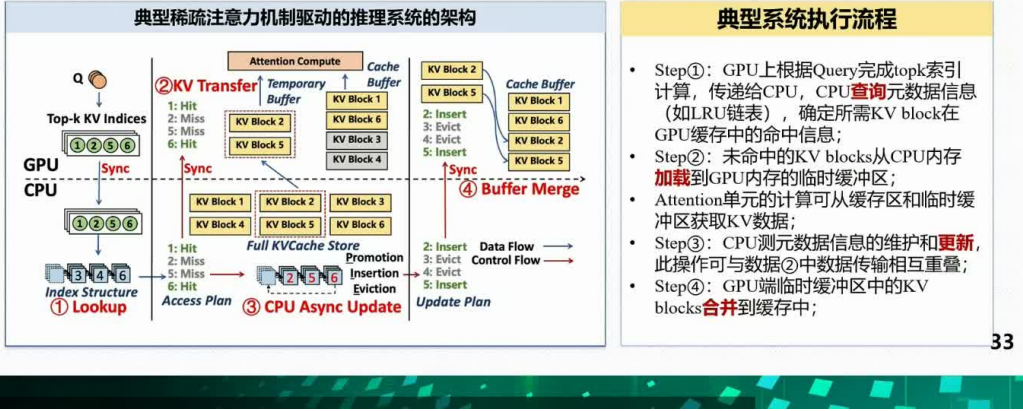


图 7: 层次化 KVCache 系统把全量 KV 放在 CPU memory，并在 GPU 上维护热点缓存和四步执行路径。⁷

1. GPU 侧计算当前 step 需要访问的 top-k KV index。
2. CPU 侧查询热点缓存 metadata，判断哪些 KV 命中 on-GPU cache。
3. 对未命中部分执行 CPU→GPU 异步加载，并更新 metadata。
4. 将命中 KV 与加载 KV 合并，进入 attention 计算。

可以用集合符号把这条路径写得更清楚。设当前 step 需要的 top-k 集合为 S_t ，GPU cache 中已有的 KV index 集合为 C_t ，则：

$$H_t = S_t \cap C_t, \quad M_t = S_t \setminus C_t$$

其中：

- H_t : cache hit 集合，表示当前 step 需要且已经在 GPU cache 中的 KV。
- M_t : cache miss 集合，表示当前 step 需要但必须从 CPU memory 加载的 KV。
- S_t : 稀疏 attention 或 KV 检索算法给出的 top-k KV index。
- C_t : 系统当前维护的 on-GPU cache index 集合。

若 $|M_t|$ 很大，系统会重新落入 PCIe 搬运瓶颈；若 $|H_t|$ 很大，系统仍要付出 metadata lookup、update 和 merge 的管理成本。这也是后续优化空间所在。

4.3 metadata 为什么成为隐藏成本

metadata 是缓存系统的“目录”和“账本”，包括某个 KV block 是否在 GPU、位于哪个位置、最近是否被访问、链表或计数器如何更新、加载后的数据怎样和命中数据合并。传统缓存系统常使

用 LRU/LFU 等策略维护热点数据；在层次化 KVCache 中，这些策略需要和 GPU 并发执行、CPU 查表、PCIe 搬运共同协作。

一次 step 的时间可以粗略拆成：

$$T_{\text{step}} \approx T_{\text{topk}} + T_{\text{lookup}} + T_{\text{copy}} + T_{\text{update}} + T_{\text{merge}} + T_{\text{attn}}$$

其中：

- T_{topk} ：计算或检索 top-k KV index 的时间。
- T_{lookup} ：查询 metadata、判断 hit/miss 的时间。
- T_{copy} ：miss KV 从 CPU memory 搬到 GPU HBM 的时间。
- T_{update} ：KV 被访问或加载后更新 metadata 的时间。
- T_{merge} ：把 hit KV 与新加载 KV 合并成 attention 输入的时间。
- T_{attn} ：实际 attention 计算时间。

层次化系统的真实瓶颈不只在数据搬运

CPU offload 加 sparse attention 看起来已经减少了 HBM 容量压力和 value 读取量，但系统仍要完成 top-k、lookup、copy、update、merge。后续 profiling 之所以重要，是因为这些“管理动作”可能占据端到端 decoding 时间的大头。

这一点连接了第三章的 warning：降低 value 读取只是路径中的一项收益。若 top-k 检索、metadata 查表和链表更新没有同步优化，稀疏化带来的节省会被管理成本吃掉。

4.4 本章小结

- 层次化 KVCache 把全量历史放在 CPU memory，把热点 KV 保留在 GPU HBM。
- 每个 decoding step 先得到 top-k KV index，再通过 metadata 判断 hit/miss。
- 未命中的 KV 需要 CPU→GPU 异步加载，随后更新 metadata，并与命中 KV 合并。
- T_{step} 由 top-k、lookup、copy、update、merge 和 attention 多部分组成，后续优化必须逐项处理。
- 下一章应聚焦现有系统中的两个核心问题：metadata 管理开销和 CPU-GPU 协同对 CUDA Graph 友好性的破坏。

5 现有系统慢在哪里：metadata 与调度

前一章把层次化 KVCache 系统拆成四步：计算 top-k index、查 metadata、加载 miss 的 KV、更新并合并结果。这个流程的初衷很清楚：把全量 KVCache 放到 CPU memory，用 on-GPU cache 保留热点 KV，从而缓解 HBM 容量压力。问题也正是在这里出现。系统把容量问题转化成了两个新的工程问题：metadata 管理成本和 CPU-GPU 协同成本。

5.1 问题一：动态 KVCache 管理开销过高

讲者在 profiling 中强调，现有系统里动态 KVCache 管理的开销已经达到端到端时间中很显眼的比例。字幕给出的观察是：在 A40 和 H100 级 GPU 上，lookup、数据加载、update、merge 等步骤合起来，可占到端到端推理时间约 40% 到 60% 以上。不同模型和序列长度下具体数值会变化，正文保留这个量级判断。

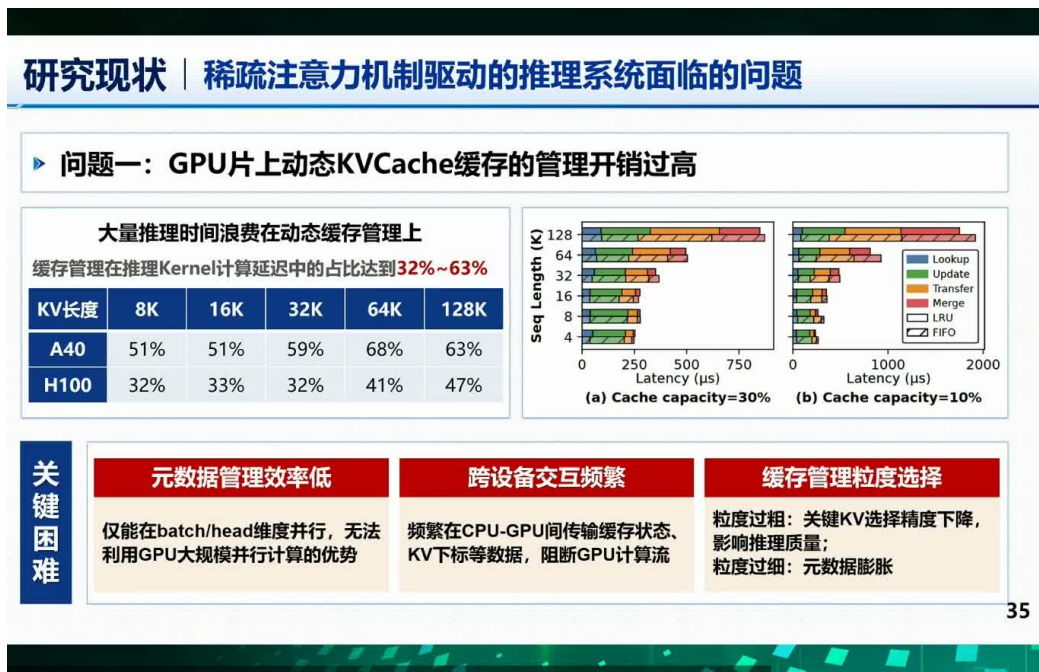


图 8: 现有系统中 lookup、load、update、merge 等管理步骤会占据显著的 Decoding latency。⁸

可以用一个简化的时间账本理解这件事。一次 decoding step 的可见时间近似写成：

$$T_{\text{step}} \approx T_{\text{topk}} + T_{\text{lookup}} + T_{\text{copy}} + T_{\text{update}} + T_{\text{merge}} + T_{\text{attn}} + T_{\text{sched}}.$$

其中：

- T_{topk} ：在当前 step 中找出需要访问的 top-k KV index 的时间。
- T_{lookup} ：根据 metadata 判断哪些 KV 已在 on-GPU cache 中命中的时间。
- T_{copy} ：从 CPU memory 通过 PCIe 搬运 miss KV 到 GPU 的时间。
- T_{update} ：缓存状态变化后更新 metadata 的时间。
- T_{merge} ：把命中 KV 与新加载 KV 合并成 attention 输入的时间。
- T_{attn} ：真正的 GPU attention 计算时间。
- T_{sched} ：kernel launch、同步、调度碎片等额外开销。

若把 metadata 相关成本粗略记为：

$$T_{\text{meta}} = T_{\text{lookup}} + T_{\text{update}} + T_{\text{merge}}, \quad \rho_{\text{meta}} = \frac{T_{\text{meta}}}{T_{\text{step}}},$$

⁸视频画面时间区间：00:08:20–00:10:10。若为裁剪图，时间区间仍对应原视频画面。

那么讲者想说明的核心就是： ρ_{meta} 已经大到足以改变系统优化方向。缓存命中率很高当然有价值，可一旦 metadata 的查、改、合并本身过重，命中带来的收益就会被管理成本吞掉。

Profiling 给出的硬结论

长上下文 KVCache offload 系统的瓶颈已经从“有没有足够 HBM 放 KV”扩展到“为了管理 on-GPU cache 付出了多少时间”。链表式 metadata、miss 数据加载、缓存更新和合并操作合起来，可能占据 decoding latency 的大头。这个事实解释了为什么单纯提高缓存命中率仍会跑得慢。

5.2 LRU/LFU 与 GPU 并发执行的结构冲突

LRU (Least Recently Used) 和 LFU (Least Frequently Used) 是经典缓存策略。它们在 CPU 软件系统中常见：LRU 像给每个缓存块排一个“最近使用队列”，新访问的条目移到队头，久未使用的条目逐步滑向队尾；LFU 像给每个条目录使用次数，频率低的条目更容易被淘汰。

这个类比帮助理解问题：队列、计数器、链表指针、位置映射都属于 metadata。传统缓存系统靠精细 metadata 换取较好的替换决策。长上下文 KVCache 系统面对的是 GPU 上大量 head、token、batch 的并行执行。GPU 擅长做规则、批量、连续的向量计算；链表和频繁条件更新更像分叉很多的办公室流程。每个节点都要查状态、改指针、维护一致性，CPU 可以处理，GPU 并行单元会被这种控制流拖慢。

把 LRU/LFU metadata 放在 CPU 管理也会付出代价。CPU 侧一次线性扫描或链表更新看似很小，放到每个 decoding step、每个请求、每个 head 的频繁路径里，就会变成可观延迟。更麻烦的是，CPU 管理 metadata 意味着 GPU 计算路径要频繁等待 CPU 给出状态判断。

为什么经典缓存策略在这里失灵

LRU/LFU 的设计背景更接近“少量核心维护一个共享缓存目录”。长上下文推理更像“成千上万个并行工作单元同时索要不同片段”。前者可以接受复杂 metadata 换取精确替换；后者更需要可并行、可批处理、可被 GPU kernel 直接执行的轻量规则。

5.3 缓存粒度：粗粒度牺牲选择质量，细粒度膨胀 metadata

讲者还指出一个更隐蔽的折中：缓存粒度会直接改变系统成本。设上下文长度为 L ，缓存 block 大小为 b ，则需要管理的 block 数量可近似写成：

$$N_{\text{block}} \approx \left\lceil \frac{L}{b} \right\rceil.$$

其中：

- L ：当前上下文 token 长度。
- b ：一个缓存 block 包含的 token 数。
- N_{block} ：metadata 需要跟踪的缓存条目数。

当 b 变大时，metadata 条目数下降，管理更轻；代价是每次选择的 KV 变粗，top-k 可能带上许多相关性较弱的 token，选择质量下降。当 b 变小时，系统能更细地挑出高质量 top-k KV；代价是 metadata 条目膨胀，lookup 和 update 更频繁。

这个折中与稀疏注意力的目标形成张力。稀疏注意力希望“只读真正相关的历史”；粗粒度 block 会把“相关”和“邻近但低相关”的 token 打包到一起。细粒度 block 更接近理想选择，却把系统拖回 metadata 管理泥潭。

常见误解：缓存命中率高就等于系统快

命中率只是缓存系统的一个指标。端到端速度还取决于 metadata 查表、更新、合并、CPU-GPU 同步、PCIe 搬移和 kernel 调度。若命中需要昂贵链表维护和频繁同步，系统仍会慢。

5.4 问题二：CPU-GPU 交互破坏 CUDA Graph 捕获

第二个核心问题是 CPU-GPU 协同。讲者特别提到，在 H100 这类高端 GPU 上，CUDA Graph 是 decoding 延迟优化中的关键选项。CUDA Graph 的直觉很简单：若一段 GPU 执行流程反复出现，并且 kernel 拓扑相对稳定，系统可以提前捕获这段图，后续复用图执行，降低 kernel launch 和调度开销。

层次化 KVCache 系统把 sparse attention、CPU offload、metadata 管理揉在一起之后，GPU 计算路径会频繁等待 CPU 的查表、加载、更新通知。原本可以被批量捕获的一组 GPU kernel 被拆成很多细粒度片段；CPU 操作插入其中，CUDA Graph 难以覆盖完整路径。对 decoding 这种逐 token、低延迟敏感的阶段，这种调度碎片化会直接拉长生成时间。

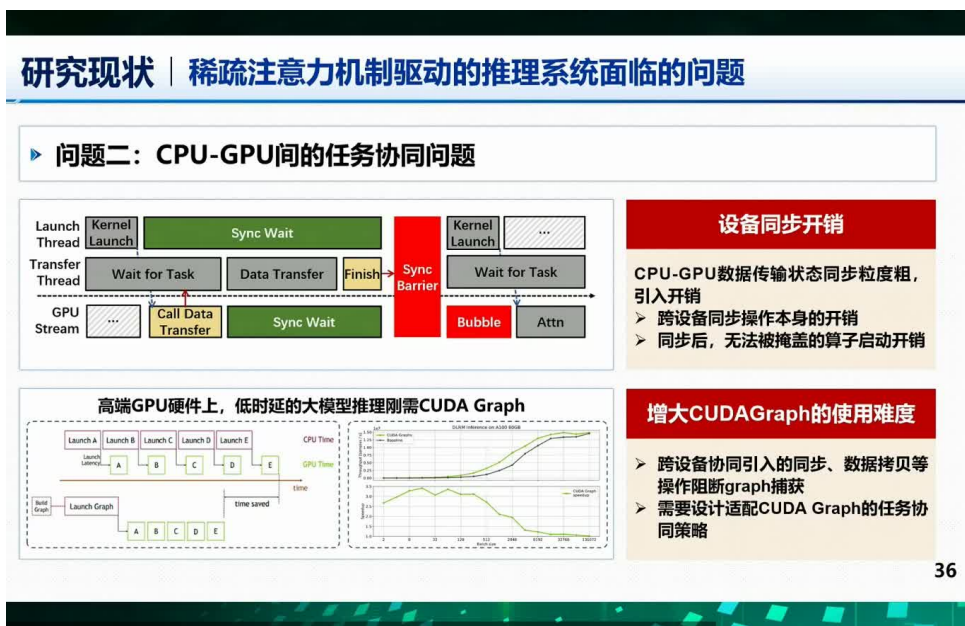


图 9: CPU-GPU 频繁同步会制造等待气泡，并增加 CUDA Graph 捕获完整 Decoding 路径的难度。⁹

两个科学问题

讲者把现有系统的瓶颈压缩成两件事：第一，GPU 侧如何维护一个开销足够低的轻量缓存算法；第二，CPU 与 GPU 之间如何完成高效协同，减少 CPU 对 GPU 执行路径的打断。

⁹视频画面时间区间：00:10:20–00:11:10。若为裁剪图，时间区间仍对应原视频画面。

¹⁰视频画面时间区间：00:11:10–00:11:28。若为裁剪图，时间区间仍对应原视频画面。

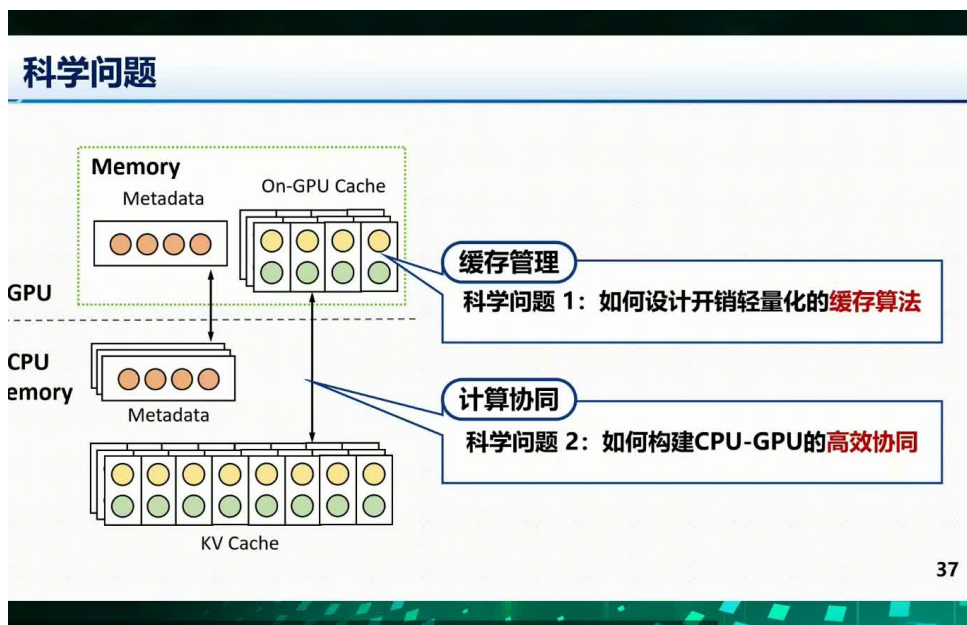


图 10: 讲者将系统瓶颈压缩为两个科学问题：轻量化缓存管理与 CPU-GPU 高效协同。¹⁰

5.5 本章小结

- 现有层次化 KVCache 系统缓解了 HBM 容量压力，同时引入 metadata 管理和 CPU-GPU 协同成本。
- Profiling 显示 lookup、load、update、merge 等管理步骤可能占据端到端 decoding 时间约 40% 到 60% 以上，具体比例随实验设置变化。
- LRU/LFU 等链表式策略与 GPU 高并发执行特性冲突；放到 CPU 管理又带来频繁同步和扫描成本。
- 缓存粒度存在工程折中：粗粒度降低 metadata 数量，细粒度提高选择质量。
- CUDA Graph 需要稳定、可捕获的执行路径；频繁 CPU-GPU 交互会把 GPU 执行切成碎片。

6 轻量缓存算法：用 query 相似性替代链表 metadata

上一章的问题逼着系统设计回到模型行为本身：若 LRU/LFU 等经典策略的 metadata 过重，就需要寻找更适合 GPU 执行的轻量信号。讲者提出的核心观察是，相邻 decoding step 的 query 往往高度相似；当 query 相似时，它们选中的 top-k KV 集合也往往高度重合。由此可以把“维护复杂链表”改成“比较当前 query 与上一轮 query 的相似度”。

6.1 观察：相邻 decoding step 的 query 高度相似

在自回归 decoding 中，模型每一步只生成一个新 token。相邻两个 step 的语义状态通常连续，比如“今天天气怎么样”里相邻词对应的上下文状态并非剧烈跳变。讲者展示了多个模型上的统计图：相邻 step 之间 query similarity 较高，热力图的对角线区域更红，说明相邻位置的 query 更接近。

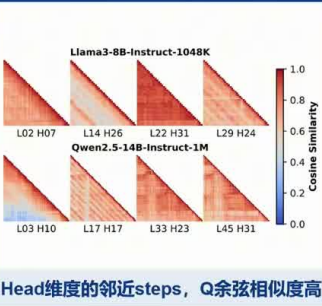
¹¹ 视频画面时间区间：00:11:30-00:14:10。若为裁剪图，时间区间仍对应原视频画面。

研究方案1 | 稀疏语义感知的轻量化缓存算法设计

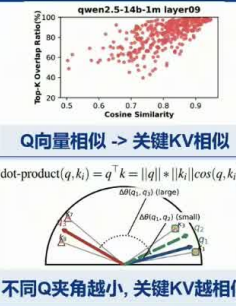
核心思想

基于Query相似性，复用关键KVCache的轻量级缓存算法

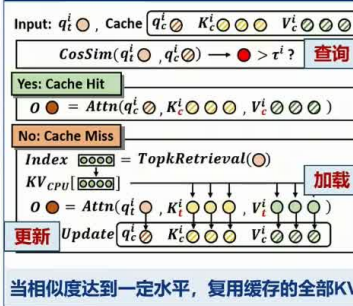
观察：邻近steps的Q向量方向相似



分析：Q方向影响KV选择



基于Query相似性的缓存算法设计



技术特色

大幅降低元数据量，简化元数据操作流程，降低缓存管理开销

39

图 11: 基于 query similarity 的缓存复用：相邻 step 的 query 越接近，top-k KV 集合越可能重合。

直觉上可以把 query 理解成“当前 token 带着什么问题去历史 KVCache 里找证据”。若连续两个 token 所带的问题非常接近，它们要查的历史片段也可能接近。这种稳定性是 LiteCache 类方法可用的前提。

一个直觉类比

读者连续追问“今天北京天气怎么样”和“明天北京天气怎么样”时，检索系统大概率会查看相近的历史资料：城市、天气、日期这些语义槽位高度重合。query similarity 在模型内部扮演类似信号：当前 step 和上一 step 的查询意图越接近，top-k KV 的重合概率越高。

6.2 从 query 相似到 top-k KV 重合

attention 的核心打分来自 query 与 key 的点积。对第 t 个 decoding step、第 h 个 head、历史位置 i ，缩放点积注意力可写成：

$$s_{t,i}^{(h)} = \frac{q_t^{(h)} \cdot k_i^{(h)}}{\sqrt{d_h}}.$$

其中：

- $q_t^{(h)}$ ：第 t 个 decoding step、第 h 个 head 的 query 向量。
- $k_i^{(h)}$ ：历史位置 i 、第 h 个 head 的 key 向量。
- d_h ：单个 head 的维度。
- $s_{t,i}^{(h)}$ ：当前 query 对历史 key 的 attention score。

query 相似性通常用 cosine similarity 衡量：

$$\cos(q_t^{(h)}, q_{t-1}^{(h)}) = \frac{q_t^{(h)\top} q_{t-1}^{(h)}}{\|q_t^{(h)}\|_2 \|q_{t-1}^{(h)}\|_2}.$$

其中:

- $q_{t-1}^{(h)}$: 上一 decoding step 同一 head 的 query 向量。
- $\|\cdot\|_2$: 向量的二范数。
- cosine similarity 衡量两个向量方向的接近程度, 取值越接近 1, 方向越一致。

若当前 query 与上一 query 的方向接近, 并且 query/key 的向量范数变化相对稳定, 则它们对历史 key 的排序通常也更稳定。于是当前 step 选出的 top-k KV index 集合 $S_t^{(h)}$ 可能接近上一 step 的集合 $S_{t-1}^{(h)}$:

$$\cos(q_t^{(h)}, q_{t-1}^{(h)}) \geq \tau_h \Rightarrow S_t^{(h)} \approx S_{t-1}^{(h)}.$$

其中:

- τ_h : 第 h 个 head 的 query 相似性阈值。
- $S_t^{(h)}$: 当前 step、第 h 个 head 选择的 top-k KV index 集合。
- $S_{t-1}^{(h)}$: 上一 step 对应的 top-k KV index 集合。

点积与余弦相似度的严格关系

字幕中把 QK production 解释成“只与夹角有关、与模长无关”。严格写法需要更谨慎: 点积满足 $q \cdot k = \|q\|_2 \|k\|_2 \cos \theta$, 同时包含向量范数和夹角。cosine similarity 是方向相似性的代理; 当 query/key 已归一化, 或范数变化相对稳定时, 它才更直接反映 top-k 排序的稳定性。

6.3 核心算法: 阈值命中与上一轮 top-k 复用

传统做法为每个缓存块维护链表或频率计数。讲者的算法把 GPU 上缓存的 metadata 改成上一轮 query 表征以及上一轮 top-k KV index。当前 step 到来时, GPU kernel 计算当前 query 与缓存 query 的 cosine similarity。若相似度超过阈值 τ_h , 系统认为当前 step 可以复用上一轮 top-k KV; 若低于阈值, 系统发生 miss, 再从 CPU memory 加载必要 KV。

这个逻辑可以写成:

$$c_t^{(h)} = \cos(q_t^{(h)}, q_c^{(h)}), \quad m_t^{(h)} = \mathbf{1}[c_t^{(h)} \geq \tau_h],$$

$$S_t^{(h)} = \begin{cases} S_c^{(h)}, & m_t^{(h)} = 1, \\ \text{TopK}_i(s_{t,i}^{(h)}), & m_t^{(h)} = 0. \end{cases}$$

其中:

- $q_c^{(h)}$: 缓存中保存的上一轮或候选 query 表征。
- $c_t^{(h)}$: 当前 query 与缓存 query 的 cosine similarity。

- $m_t^{(h)}$: 相似性命中标记, 1 表示复用, 0 表示重新选择并加载。
- $S_c^{(h)}$: 缓存中可复用的上一轮 top-k KV index 集合。
- $\text{TopK}_i(\cdot)$: 对历史位置 i 按 attention score 选择 top-k。

命中时, 系统直接复用上一轮 top-k KV, 避免复杂 lookup 和链表更新。miss 时, 数据加载路径仍存在: CPU memory 中的 KV 需要通过 PCIe 进入 GPU。更新操作也变轻, 主要变成替换缓存中的 query 向量和相关轻量状态。

算法压缩

该方法把缓存管理从“维护 LRU/LFU 链表”压缩成“在 GPU 上比较 query 向量相似度”。它利用模型内部的连续性, 将 metadata 从复杂数据结构换成向量状态, 使 lookup 与 update 更接近 GPU 擅长的向量计算。

6.4 自适应阈值、resident head 与 GQA 聚合

一个固定阈值很难覆盖所有模型、数据集和 head。讲者指出, 不同 head 的重要性和复用率不同: 有些重要 head 应设置更严格条件, 尽量保证 top-k 选择质量; 重要性较低或相似性更稳定的 head 可以放宽条件, 获得更多复用机会。

可以把这种策略抽象成:

$$\tau_h = f(I_h, R_h, D),$$

其中:

- I_h : head h 的重要性估计。
- R_h : head h 的历史复用率或相似性稳定程度。
- D : 当前数据集、请求类型或运行时分布信息。
- f : 自适应阈值策略; 原视频给出的是机制思路, 具体实现细节不在本笔记中展开。

讲者还提到 resident head: 有些 head 特别重要, 且 query 相似度偏低, 复用率有限。对这类 head, 若 GPU HBM 允许, 可以把它们的全量 KVCache 常驻在 GPU 中, 减少错误复用风险和反复加载成本。

resident head 的意义

resident head 像给关键岗位留固定工位。普通 head 通过 query similarity 复用上一轮 top-k; 少数关键且不稳定的 head 直接保留完整 KV, 从而用显存换取可靠性。这个设计承认不同 attention head 的行为差异, 避免把所有 head 塞进同一条规则。

Grouped-Query Attention (GQA) 进一步带来聚合问题。在 GQA 中, 多个 query head 共享一组 KV head, 因此系统需要把同一 KV group 内多个 query head 的重要性和相似性汇总到 group 级决策上。原视频没有给出可直接引用的精确聚合公式, 本笔记只描述“组内聚合”的机制: 重要 query head 对精度更敏感, 低相似度 head 更需要保守处理, 而共享 KV head 的 group 需要把这些信号合并后再决定是否复用缓存。

GQA 聚合的边界

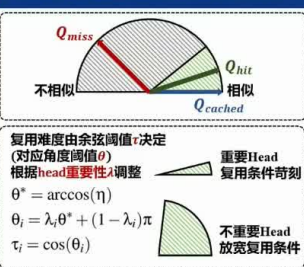
这里不能把“组内聚合”写成固定公式。幻灯片展示的是机制设计：比较、聚合、再决定是否复用。真正的聚合函数属于实现细节，若没有论文或代码支撑，写成精确数学公式会制造伪精确。

研究方案1 | 稀疏语义感知的轻量化缓存算法设计

方案细节

感知Head重要性（对精度的贡献），完善缓存策略

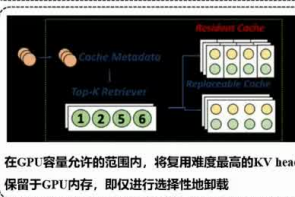
缓存效率与推理精度间的权衡方案



对推理精度越重要，复用控制越严格

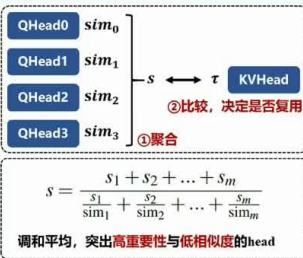
重要性高但相似度低的数据处理策略

复用难度：衡量head的相似度水平与重要性水平的diff
 $D_i = \tau_i - (\hat{s}_i - \varepsilon)$



提高系统性能稳定性

面向GQA模型的组内相似性聚合方法



适配主流LLM体系结构

技术特色

维持精度的前提下最大程度提高缓存命中率，提高推理效率

40

图 12: 自适应阈值、resident head 与 GQA 聚合共同处理不同 head 的重要性和复用稳定性差异。¹²

6.5 本章小结

- 相邻 decoding step 的 query 往往相似；query 越相似，top-k KV 集合越可能重合。
- cosine similarity 衡量方向接近程度；点积还包含向量范数，正文需保留这个条件。
- 核心算法用 query 相似性命中替代 LRU/LFU 链表 lookup，命中时复用上一轮 top-k KV，miss 时再从 CPU memory 加载。
- 自适应阈值把不同 head 的重要性和稳定性纳入缓存决策；resident head 用显存保留少数重要且低复用的 head。
- GQA 场景需要把多个 query head 的重要性与相似性汇总到共享 KV group；本笔记保留机制解释，避免编造实现细节。

7 协同调度：让数据搬移尽量隐藏在计算之后

query 相似性复用降低了 metadata 成本，也减少了一部分 miss。可它没有改变一个物理事实：只要 miss 发生，相关 KV 就必须从 CPU memory 经过 PCIe 进入 GPU。第七章讨论的是第二条

¹²视频画面时间区间：00:14:20-00:15:30。若为裁剪图，时间区间仍对应原视频画面。

研究主线：在数据仍需移动的前提下，系统怎样把可见搬移时间压低，并让 GPU 执行路径重新变得 CUDA Graph 友好。

7.1 miss 数据仍需搬运：算法无法消灭物理带宽

CPU offload 的收益来自容量扩展：CPU memory 能承载更大的 KVCache。它的代价来自互连带宽：PCIe 的有效带宽远低于 HBM。query 相似性复用可以减少 miss 概率，降低需要从 CPU 拉取的 KV 数量；剩余 miss 的字节仍要穿过 PCIe。

不要把复用误读成零搬运

LiteCache 类算法降低的是无效管理与部分 miss 数据搬运。对实际发生的 miss，系统仍受 BW_{PCIe} 、CPU memory 访问路径、GPU 接收缓冲和同步机制影响。工程优化的目标是减少可见延迟，物理数据移动仍然存在。

可见搬移时间可以用一个简化式表达：

$$T_{\text{copy,visible}} \approx \max(0, T_{\text{copy}} - T_{\text{compute,overlap}}).$$

其中：

- T_{copy} ：miss KV 从 CPU 到 GPU 的数据搬移时间。
- $T_{\text{compute,overlap}}$ ：可与数据搬移重叠的 GPU 片上计算时间。
- $T_{\text{copy,visible}}$ ：用户在端到端 latency 中实际感受到的搬移残余。

这个式子的含义很朴素：数据搬移时间若能被 GPU 计算覆盖，端到端延迟就只暴露覆盖之后的残余。若计算时间不足以覆盖搬移，剩下的 PCIe 时间仍会成为瓶颈。

7.2 与 InfiniGen 类 overlap 思想的结合

讲者提到 OSDI 2024 的 InfiniGen 类路线，用相邻 layer 之间 query 的相似性来安排数据传输与 GPU 片上计算的 overlap。当前方法使用相邻 decoding step 的 query 相似性，天然可以与这种 overlap 思路结合：提前判断哪些 KV 可能要用，把可预测的数据搬移放到可重叠窗口中。

可以把整个路径理解成三段流水线：

1. GPU 侧基于 query similarity 判断当前或后续 step 的 KV 复用情况。
2. 对 miss 或预测 miss 的 KV，CPU memory 提前发起搬移。
3. GPU 在等待期间继续执行可并行的片上计算，尽量把 T_{copy} 藏进 T_{compute} 。

overlap 的核心收益

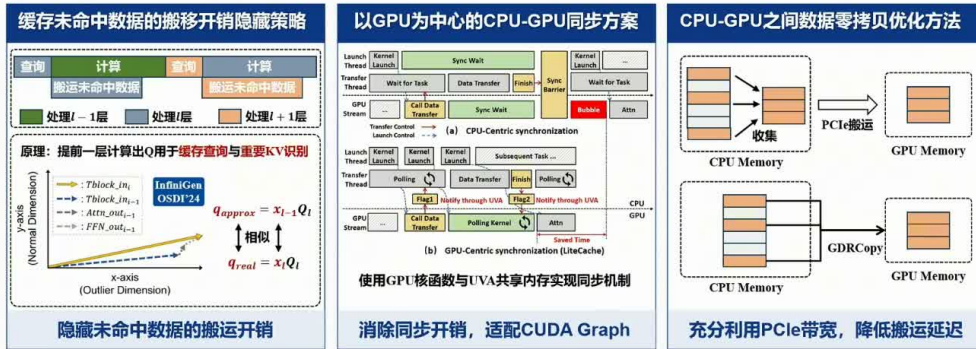
overlap 并没有提高 PCIe 的物理上限。它改变的是时间排布：把“先等数据再计算”的串行路径改成“搬数据时执行可独立计算”的流水路径。端到端吞吐提升来自等待时间被覆盖。

¹³ 视频画面时间区间：00:15:40–00:17:10。若为裁剪图，时间区间仍对应原视频画面。

研究方案2 | 稀疏注意力友好的CPU-GPU协同机制

方案细节

从数据搬移、流量控制、计算与传输掩盖等角度设计协同机制



技术特色

以GPU为中心的全方位、多层次优化方案，追求极致性能

41

图 13: CPU-GPU 协同方案从数据搬移、同步粒度和额外拷贝三个角度降低可见开销。¹³

7.3 GPU 触发 CPU：恢复 CUDA Graph 友好执行

上一章说明了 CPU 频繁打断 GPU 会让 CUDA Graph 难以捕获完整 decoding 路径。讲者的解决方向是反转控制关系：减少 CPU 主动插入 GPU 执行流，让 GPU 侧 kernel 触发或通知 CPU 完成必要协同。这样，相关同步动作在 GPU 视角下更像一个可捕获的 kernel 节点，CUDA Graph 更容易覆盖稳定路径。

CUDA Graph 为什么在 decoding 中重要

decoding 是逐 token 重复执行的低延迟路径。每一步都发起大量小 kernel 时，launch 和调度开销会变得显眼。CUDA Graph 把固定执行图预先捕获，后续重复提交图，从而降低 CPU 侧调度参与度。若 CPU 在中间频繁做 metadata 决策，图捕获范围会被切碎。

工程上，这类设计有两个关键要求。第一，GPU 侧触发逻辑要足够轻，不能为了规避 CPU 打断而引入更重的 kernel。第二，CPU 与 GPU 共享的状态必须保持可控，避免把同步点变成新的全局阻塞。

7.4 GDRCopy 与额外拷贝消除

讲者还提到 GDRCopy。GDRCopy 是 NVIDIA 生态中用于降低 GPU 与 CPU 之间小块数据访问和拷贝开销的机制。这里关注它在数据路径上的作用：减少 CPU 侧额外复制，使 PCIe 数据搬移更接近真正需要的字节传输。

在普通实现中，数据可能经历“CPU memory 中的一份数据 -> 中间 buffer -> GPU 接收 buffer”的额外周转。每多一次 copy，系统就多消耗 CPU memory bandwidth、缓存污染和同步成本。GDRCopy 类机制的价值在于缩短数据路径，提高 PCIe 有效利用率。

第七章的工程主线

这一章解决的是 residual miss。query similarity 降低 miss 与 metadata 成本；overlap 让必要搬移尽量隐藏在计算后面；GPU 触发 CPU 减少调度碎片；GDRCopy 降低额外 copy。四者共同把 CPU offload 从“容量补丁”推进到可用的长上下文推理路径。

7.5 本章小结

- query 相似性复用不能消灭所有 miss；miss KV 仍需通过 PCIe 从 CPU memory 进入 GPU。
- overlap 的目标是降低 $T_{\text{copy,visible}}$ ，通过可重叠 GPU 计算覆盖部分数据搬移时间。
- 与 InfiniGen 类思想结合时，相邻 query 的稳定性可用于安排提前搬移和流水化执行。
- GPU kernel 触发或通知 CPU 的协同路径有助于减少 CPU 打断，使 CUDA Graph 捕获更可行。
- GDRCopy 通过减少 CPU 侧额外 copy，提高 residual miss 数据路径的有效带宽利用。

8 实验评估：性能、容量、精度和显存

实验部分要回答一个朴素问题：这个系统是否真的把前文诊断出的 metadata 管理、CPU-GPU 协同、容量扩展和精度保持问题压下去了。评价维度需要覆盖 decoding throughput、可支持上下文长度、metadata 开销、LongBench/RULER 精度、GPU 显存占用，以及 Prefill 阶段是否引入额外损失。

8.1 实验平台与基线系统

字幕显示实验在 H100 和 A40 级 GPU 上进行，互连分别对应 PCIe 5.0/4.0 一类平台背景；具体配置以清晰实验页为准。讲者使用三类 baseline：

- FullAttn (FullAttention, 全量注意力基线)：全量 KVCache 保留在 HBM 中，作为无 offload、无稀疏化的参照。若该方法吞吐保持稳定，说明 CPU-GPU 搬移开销已被有效隐藏或降低。
- InfiniGen：KVCache offload 与检索相关 baseline。
- RetroInfer：KVCache offload/稀疏化相关强 baseline。

为什么 FullAttn 是关键参照

FullAttn 把 KVCache 全量放在 HBM 中，绕开 CPU offload 的 PCIe 搬运问题。它的限制是显存容量。若 offload 系统在可支持更长上下文的同时保持接近 FullAttn 的吞吐和精度，说明系统已经把容量扩展、检索选择和数据搬移三件事协调起来。

¹⁴视频画面时间区间：00:17:20–00:18:10。若为裁剪图，时间区间仍对应原视频画面。

研究结果 | 实验评估

- ▶ 硬件平台：① NVIDIA H100 GPU x1 + PCIe 5.0; ② NVIDIA A40 GPU x1 + PCIe 4.0
- ▶ Baselines:
 - ▶ FullAttn: 无KVCache卸载, 无稀疏注意力, 采用SGLang的实现
 - ▶ InfiniGen: KVCache卸载 + 稀疏注意力 + 预取关键KV
 - ▶ RetroInfer: SOTA, KVCache卸载 + 稀疏注意力 + GPU片上动态缓存关键KV

Decode端到端性能相比RetroInfer取得1.19x~3.24x加速比, 且可支持的最大token数更多

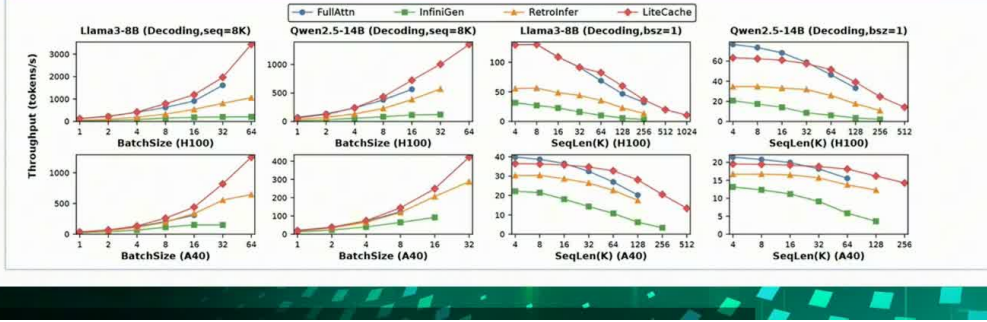


图 14: 实验以 FullAttn、InfiniGen、RetroInfer 等作为 baseline, 比较 Decoding 吞吐和可支持上下文长度。¹⁴

8.2 decoding throughput 与可支持上下文长度

讲者的结论是: 随着序列长度或 batch size 增大, 该方法在 decoding 阶段相比最强 baseline 有吞吐提升, 并且可支持的 token 数更多, 最高展示到约 1M token 量级。由于实验图中的具体倍数和曲线文字较密, 本节采用保守写法, 只给出有画面和字幕共同支撑的定性结论。

decoding throughput 可抽象为:

$$\text{Throughput}_{\text{decode}} = \frac{N_{\text{generated}}}{T_{\text{decode}}}$$

其中:

- $N_{\text{generated}}$: 实验时间窗口内生成的 token 数。
- T_{decode} : 对应的 decoding 总耗时。

可支持上下文长度则受到 HBM 中常驻内容限制。一个简化约束是:

$$M_{\text{HBM}} \geq M_{\text{model}} + M_{\text{workspace}} + M_{\text{onGPUCache}}$$

其中:

- M_{HBM} : 可用 GPU 显存容量。
- M_{model} : 模型权重及必要运行时状态占用。
- $M_{\text{workspace}}$: 临时 workspace、kernel buffer 等运行时占用。
- $M_{\text{onGPUCache}}$: 保留在 GPU 上的热点 KVCache 或 resident head KVCache。

若 query similarity 复用和更轻 metadata 让 $M_{\text{onGPU}Cache}$ 更小, 系统就能把更长历史放到 CPU memory, 并在 GPU 上保留更少、更关键的 KV 子集。

实验结论的保守写法

正文采用保守实验表述: 该方法在 decoding throughput 上超过强 baseline, 并展示更长可支持上下文长度。具体加速倍数不从压缩帧或 ASR 字幕中推断。

8.3 metadata 开销拆解

实验拆解直接回到第五章的问题: 传统系统的 metadata 管理开销较高, 而 query similarity 复用后, lookup/update/merge 这类管理开销显著下降。讲者的表述是, 最后基本只剩 GPU 上的计算时间和必要数据移动。

这说明方法的收益并非单一来源。吞吐提升可能同时来自:

- query 相似性命命中减少 CPU memory 拉取。
- query 向量 metadata 替代 LRU/LFU 链表, 降低 lookup 和 update。
- CPU-GPU overlap 降低 $T_{\text{copy,visible}}$, 也就是端到端路径中暴露出来的残余搬移时间。
- GPU 触发 CPU 与 CUDA Graph 友好路径降低 T_{sched} 。

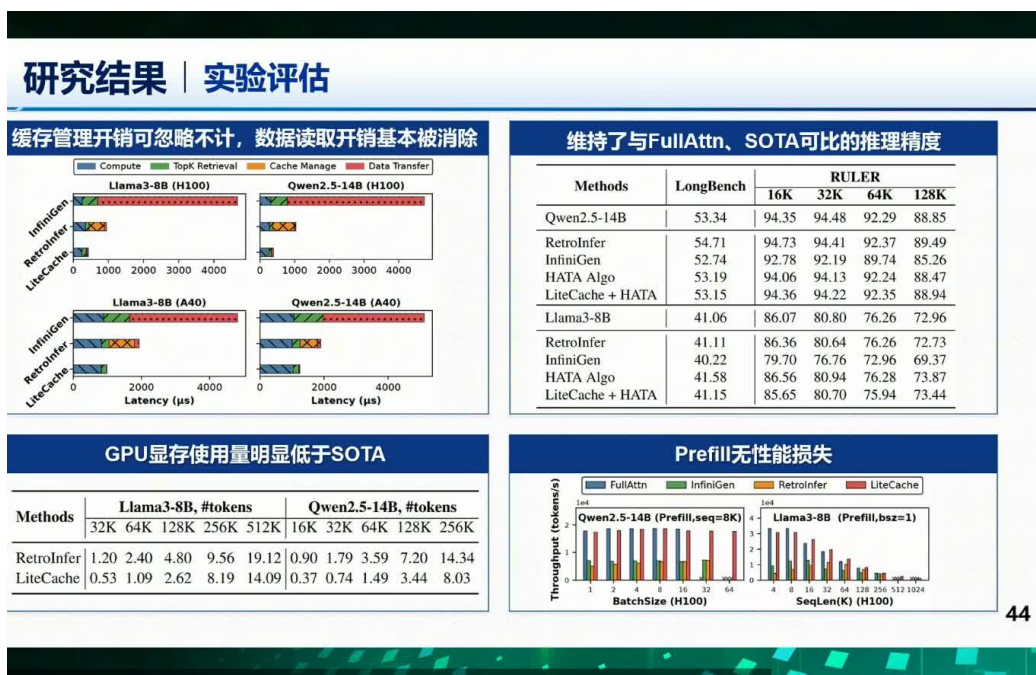


图 15: 实验结果覆盖缓存管理开销、LongBench/RULER 精度、GPU 显存使用和 Prefill 性能影响。

¹⁵ 视频画面时间区间: 00:18:20-00:19:10。若为裁剪图, 时间区间仍对应原视频画面。

8.4 LongBench/RULER 精度与 FullAttn 可比性

长上下文稀疏化最容易被质疑的是精度。若 top-k KV 复用过于激进，模型可能错过关键历史信息。讲者在 LongBench 和 RULER 上做了精度测试，并给出与 FullAttn 可比的结论。

这个结论需要严格限定：可比性来自实验设置下的 benchmark 结果，不能推成所有任务、所有 agent 场景都安全。尤其在 multi-agent 工作流中，工具返回、角色切换、任务目标突变会让 query 稳定性下降。第九章会把这一点放进适用边界。

精度结论的边界

“与 FullAttn 可比”只能按实验图和 benchmark 设置理解。query 相似性是统计规律，遇到任务跳变、工具结果插入、长距离精确信息检索时，top-k 复用可能带来遗漏风险。最终系统需要运行时监控或 fallback 策略。

8.5 GPU 显存占用与 Prefill 影响

讲者还强调，该方法相比 RetroInfer 类 baseline 使用更少 on-GPU cache 空间。这一点和第六章的 resident/head 选择有关：系统把 GPU 空间留给更有价值的 KV 子集或重要 head，减少额外缓存占用，使显存受限场景能支持更长上下文。

对 Prefill 阶段，讲者给出的结论是基本无性能损失。原因很直接：方案主要作用在 decoding 阶段的 KVCache 复用、metadata 管理和 CPU-GPU 协同路径；Prefill 仍然是对输入序列的一次性并行计算，方案没有在 Prefill 路径引入额外重操作。

实验回答了什么

实验从四个方向支撑前文机制：decoding throughput 提升，说明 metadata 与调度成本下降；最长上下文增加，说明 CPU offload 与更小 on-GPU cache 生效；LongBench/RULER 精度可比，说明 top-k 复用没有明显破坏 benchmark 表现；Prefill 无额外损失，说明优化集中在 decoding 路径。

8.6 本章小结

- 实验平台包含 H100 与 A40 级 GPU，实验图还给出 PCIe、模型和 batch 等设置。
- 基线包括 FullAttn、InfiniGen、RetroInfer；字幕中的误识别拼写不能进入最终正文。
- 该方法在 decoding throughput 上相比强 baseline 有提升，并展示约 1M token 级上下文支持；具体倍数需由实验图确认。
- metadata 拆解支持第五章诊断：传统系统管理开销高，query similarity 复用显著压低 lookup、update、merge 成本。
- LongBench/RULER 精度与 FullAttn 可比，Prefill 阶段基本无额外损失；这些结论都应绑定到实验设置。

9 总结与延伸

9.1 一句话压缩

这场分享的核心，是把 multi-agent 长上下文推理中的 KVCache 问题拆成容量、检索和调度三层：容量层用 CPU offload 承接超长历史，检索层用 sparse attention 与 top-k KV 选择减少无效访问，调度层用 query similarity 复用和 CPU-GPU 协同降低 metadata 与 PCIe 可见成本。

全片主线

长上下文管理已经进入系统层。前端 summary/compaction 负责整理“给模型哪些历史语义”；推理框架内部必须解决“KVCache 如何存、取、搬、算”。本视频集中处理后一个问题，并给出 query similarity 复用加 CPU-GPU overlap 的系统方案。

9.2 三层问题：容量、检索、调度

第一层是容量。KVCache 随上下文长度 L 、batch size B 、层数、KV head 数和 head 维度线性增长。HBM 放不下时，CPU memory offload 成为自然选择。容量层的关键词是：

- HBM 容量限制。
- CPU memory offload。
- PCIe 互连带宽。

第二层是检索。CPU memory 里可以放更多 KV，可 decoding 时不能把所有历史 KV 都搬回 GPU。sparse attention 的核心是只选当前 step 真正需要的 top-k KV。检索层的关键词是：

- QK score。
- top-k KV index。
- 训练无关 KV 检索与 trainable sparse attention。

第三层是调度。即便 top-k 已经选出，系统还要完成 metadata lookup、miss load、update、merge、CUDA Graph 捕获、GDRCopy 数据路径等工程环节。调度层的关键词是：

- metadata overhead。
- query similarity reuse。
- CPU-GPU coordination。
- CUDA Graph 与 GDRCopy。

三层视角的价值

容量、检索、调度分别回答不同问题：放在哪里、读哪一部分、怎样让搬运和计算排得更好。很多长上下文系统讨论只停留在容量或算法层，真正上线时会被第三层的 metadata 和调度成本卡住。

9.3 方法适用边界

query similarity 复用依赖一个统计前提：相邻 decoding step 的 query 足够稳定，并且 query similarity 能较好预测 top-k KV 重合。这个前提在连续文本生成、稳定主题问答、长文摘要中更容易成立。它在以下场景需要更谨慎：

- 工具调用结果突然插入，语义状态发生跳变。
- multi-agent 角色切换，例如规划 agent、执行 agent、审查 agent 的目标函数不同。
- 任务从叙述转向精确查找，例如要求引用某个很早出现的具体数值。
- 对 safety、legal、medical 等高风险内容进行长距离条件判断。

因此，系统不应把 query similarity 命中视为无条件正确。更稳妥的实现需要 fallback：当相似度低、重要 head 不稳定、输出不确定性升高、或检索证据不足时，系统应回到更保守的 top-k 选择或 FullAttn 近似路径。

最容易遗漏的风险

multi-agent 场景的上下文增长常伴随任务状态跳变。一个 agent 生成计划，另一个 agent 执行 shell，第三个 agent 做安全审查，它们的 query 分布可能连续性较差。若系统只根据相邻 step 相似性复用 top-k KV，角色切换处可能正是最需要重新检索历史的位置。

9.4 对 multi-agent 推理系统的启发

multi-agent 系统常被理解成提示词编排或前端 memory 管理问题。这个视频提醒读者：当上下文长度进入数十万乃至百万 token 量级后，瓶颈会下沉到推理框架内部。一次请求能否跑完，取决于 KVCache 的容量账本；一次请求是否快，取决于历史 KV 的检索与搬运；一次请求是否稳定，取决于 metadata 和 CPU-GPU 协同路径。

对工程系统来说，可以得到三个启发：

- 前端 compaction 与后端 KVCache 管理要协同设计。前端应尽量减少无价值历史；后端应高效处理仍需保留的长上下文。
- agent 角色切换应成为 runtime 信号。角色切换、工具结果、大段外部文档插入都可以降低 query 复用阈值，触发更保守检索。
- 性能指标要覆盖端到端路径。只看 attention FLOPs 或理论 KV 减少量会漏掉 metadata、PCIe、CUDA Graph 捕获和 CPU 调度成本。

面向系统实现的判断框架

一个长上下文 multi-agent 推理系统应同时问五个问题：KVCache 放在哪里；每步读哪些 KV；metadata 由谁维护；miss 数据如何搬；GPU 执行能否被 CUDA Graph 稳定捕获。少问任何一个，系统都可能在真实负载下失速。

9.5 开放问题

本视频给出了清晰方向，也留下了值得继续研究的问题：

- query similarity 在 agent 角色切换、工具调用、错误恢复、长程规划中的稳定性如何量化。
- 阈值 τ_h 的自适应策略能否跨模型族、数据集和业务负载迁移。
- resident head 的选择是否可以在线调整，避免显存预算被静态策略锁死。
- 1M token 级上下文中，稀疏检索遗漏关键证据时如何被监控和回滚。
- 多请求、多租户、continuous batching 场景下，CPU-GPU 协同机制如何与全局调度器整合。
- 精度评测如何覆盖真实 multi-agent 工作流，而非只依赖 LongBench/RULER 等标准 benchmark。

最终，长上下文管理的难点不在单个公式，而在跨层组合：模型行为提供 query 稳定性，算法把稳定性转成 top-k 复用，系统把复用转成更轻 metadata 和更少 miss，运行时把剩余 miss 藏进 GPU 计算与高效数据路径。这个链条上任何一环过强假设，都会在真实 multi-agent 负载中暴露。

9.6 复盘检查表

读完这份笔记后，可以用下面的检查表快速判断一个长上下文推理方案是否真的覆盖了视频中的核心问题：

- 容量账是否清楚。KVCache 预算应写成 $M_{KV} = B \cdot L \cdot N_\ell \cdot N_{kv} \cdot d_h \cdot s \cdot 2$ 。这里的 2 对应 key 和 value， s 是每个元素的字节数。若方案没有说明 B 、 L 、batching 策略和 KV head 数，容量结论就缺少可复算性。
- 检索目标是否明确。top-k KV 选择要回答“为了当前 query，哪些历史 token 仍有用”。只说稀疏比例或压缩率不够，关键是遗漏关键 KV 时怎样发现、怎样回退。
- 复用前提是否被监控。query similarity 的收益来自相邻 step 的稳定性。可用 $\cos(q_t, q_{t-1})$ 、top-k overlap、miss rate 三个信号一起判断是否继续复用。
- head 级差异是否被保留。GQA 或 head aggregation 会降低管理成本，也可能掩盖某些 head 的特殊职责。resident head、阈值 τ_h 和 fallback 策略需要一起设计。
- 调度开销是否进入端到端指标。真实 latency 可以粗略写成 $T = T_{attn} + T_{meta} + T_{copy, visible} + T_{sync}$ 。若只优化 T_{attn} ，metadata、PCIe 和同步气泡仍会吞掉收益。
- multi-agent 状态变化是否成为运行时信号。角色切换、工具调用返回、大段文档插入、错误恢复都可能让 query 分布跳变。这些事件应降低复用信心，并触发更保守的检索路径。

这个检查表的作用类似系统压测前的验收清单：它迫使读者同时检查容量、算法和运行时三类约束，避免把某一层的局部改进误判成端到端长上下文能力。